POLYTECH
NICE SOPHIA

Exceptions et énumérations

Présentation : Tom Niget

Mail de contact : polytech@niget.fr

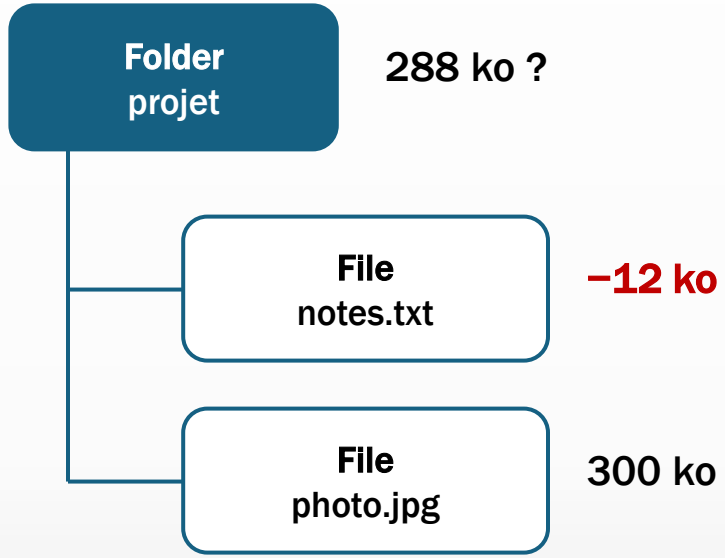
Site et ressources : niget.fr

Cours réalisé grâce notamment aux ressources de :
Julien Deantoni, Benjamin Werner, Joseph Albahari

État incohérent

```
File notes("notes.txt", -12);
File nameless("", 300);
```

Le programme compile et s'exécute.
La taille du dossier qui contient `notes.txt` sera fausse, et rien n'indiquera d'où vient l'erreur.



- En C, une fonction signale une erreur par sa **valeur de retour** (`-1`, `NULL`) : facile à utiliser mais l'appelant peut oublier de la tester !
- Un constructeur ne renvoie rien : il faut un autre moyen de **refuser la construction**

Lancement d'exceptions

```cpp
File::File(std::string name, int size_kb)
    : Entry(std::move(name)), size_kb_(size_kb) {
    if (size_kb_ < 0) {
        throw std::invalid_argument("size must not be negative"); // <stdexcept>
    }
}
```

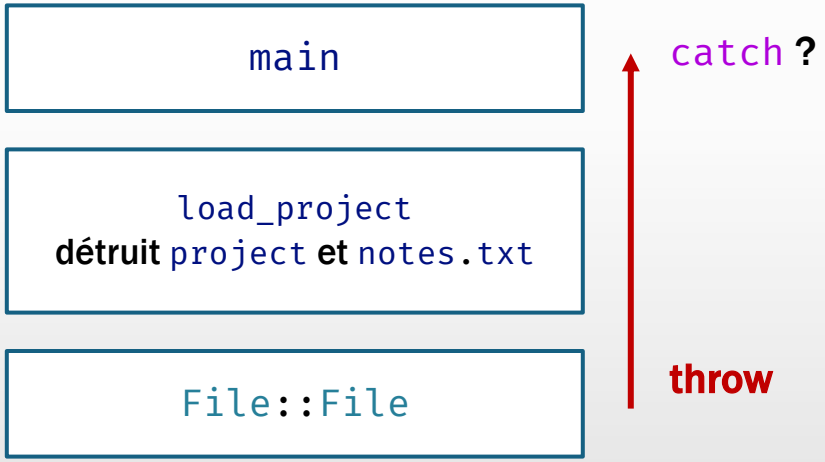
- `throw` quitte **immédiatement** la fonction (ici le constructeur)
- L'objet n'est pas créé ; le code qui suit sa déclaration n'est pas exécuté
- Ce qui était déjà construit (la partie `Entry`, son nom) est détruit automatiquement
- `~File()` n'est pas appelé car il n'y a jamais eu de fichier

Chaque classe est responsable de sa propre intégrité et doit vérifier les champs qu'elle déclare : ici `File` vérifie la taille.

Exceptions et flot non local

```
Folder load_project() {
    Folder project("projet");
    project.add(std::make_unique<File>("notes.txt", 12));
    project.add(std::make_unique<File>("photo.jpg", -1)); // lance
    return project; // jamais atteint
}
```

- L'exception quitte le constructeur de `File`, puis `load_project`, puis la fonction appelante… jusqu'à un `catch` qui lui correspond, autant que besoin (le flot d'exécution est dit « non-local »).
- À chaque fonction quittée, les objets locaux sont **détruits** : `project` détruit `notes.txt`. Ainsi, pas de fuites mémoire.



Sans `catch`, le programme s'arrête :

```
terminate called after throwing an instance of 'std::invalid_argument'
  what():  size must not be negative
Aborted (core dumped)
```

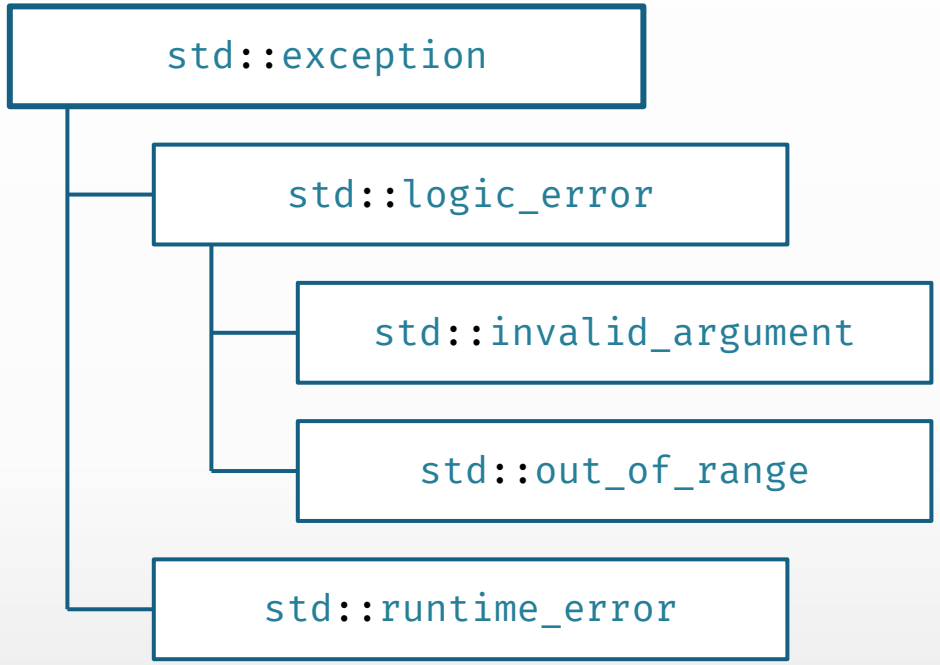
Interception d'exceptions

```
try {
    Folder project = load_project();
    std::cout << project.size_kb() << " ko\n";
} catch (const std::invalid_argument& error) {
    std::cerr << "projet illisible : " << error.what() << '\n';
}
```

- Si une exception est lancée dans le `try`, la fin du bloc est **sautée** ; l'exécution reprend dans le `catch` correspondant, puis après lui
- Interception par **référence constante**
- `what()` (**méthode héritée de** `std::exception`) **renvoie le message donné au** `throw`

Le try-catch permet notamment de laisser l'appelant décider quoi faire des erreurs qui surviennent : afficher un message, redemander une valeur, abandonner.

Exceptions standard



erreur du code appelant, évitable avant l'appel

argument refusé

indice hors limites (`vector::at`)

erreur générique découverte à l'exécution (fichier absent…)

- Un `catch` d'une classe intercepte aussi ses **classes dérivées** :
  `catch (const std::logic_error&)` **intercepte un** `invalid_argument` **ou un** `out_of_range`

Invariants et préconditions

Invariant : vrai pendant toute la vie de l'objet
Un fichier a un nom non vide et une taille positive ou nulle. Le constructeur l'établit, chaque méthode qui modifie l'objet le préserve.

Précondition : condition pour appeler une méthode
Elle porte sur un argument (l'indice de `at`) ou sur l'état de l'objet.

```
void File::resize(int size_kb) {
    if (read_only_) {
        throw std::logic_error("file is read-only");
    }
    if (size_kb < 0) {
        throw std::invalid_argument("size must not be negative");
    }
    size_kb_ = size_kb;
}
```

précondition sur l'état

préserve l'invariant

Les vérifications se font **avant** la modification. Comme ça, si une exception est lancée, l'état de l'objet est inchangé.

Types énumérés avancés

```
File config("config.ini", 4, true); // lecture seule, ou modifiable ?
```

Une chaîne serait plus lisible, mais une faute de frappe ne se remarquerait qu'à l'exécution :

```
File config("config.ini", 4, "read-onyl");
```

Une énumération permet de lister un ensemble fini de choix :

```
enum class Access { ReadOnly, ReadWrite };

File config("config.ini", 4, Access::ReadOnly);
if (access_ == Access::ReadOnly) { /* ... */ }
```

	enum du C	enum class
Noms	visibles partout : `READ_ONLY`	préfixés : `Access::ReadOnly`
Conversion en `int`	implicite	explicite : `static_cast<int>(access)`

Vérification d'exhaustivité

```
void print(const File& file) {
    switch (file.access()) {
    case Access::ReadOnly:
        std::cout << "[lecture seule] ";
        break;
    case Access::ReadWrite:
        break;
    }
    std::cout << file.name() << " : " << file.size_kb() << " ko\n";
}
```

- Sans `default`, le compilateur (avec `-Wall`) signale un `switch` qui **oublie une valeur**
- Si on ajoute `Access::WriteOnly`, il indique chaque `switch` à compléter

```
warning: enumeration value 'WriteOnly' not handled in switch [-Wswitch]
```